ONE MORE TIME

Game Design Document | Versão 0.1

Protótipo para Game Jam | Unity

2D Shooter · Bullet Hell · Temporal Echo · Boss Rush · Single-player

1. Visão geral

High concept: um shooter de nave em que o jogador grava e reproduz suas próprias ações passadas para lutar simultaneamente com versões anteriores de si mesmo, desviar de armadilhas e explorar combinações temporais para derrotar chefes.

Pilar de design: cada tentativa deve ensinar algo e criar uma nova possibilidade. O jogador não fica mais forte apenas por ter mais vida ou dano: ele aprende a coordenar suas ações com as versões anteriores da nave.

Fantasia do jogador: ser um piloto preso em um ciclo temporal que transforma cada movimento, esquiva e disparo em parte de uma estratégia maior.

Premissa de câmera: 2D side-scrolling, com a nave avançando horizontalmente, inspirada nas fases de avião de Cuphead. O tema central é One More Time: cada tentativa deixa ações para trás que podem ser reproduzidas para criar novas estratégias.

Pilares de experiência

1. O passado continua lutando

As ações gravadas tornam-se ecos que repetem trajetórias, tiros e gadgets.

2. Combate baseado em habilidade

O jogador precisa dominar a movimentação, ler padrões de ataque e criar boas trajetórias.

3. Soluções inteligentes

Armadilhas, dispositivos e chefes podem ser vencidos por sincronização e posicionamento, não apenas por dano bruto.

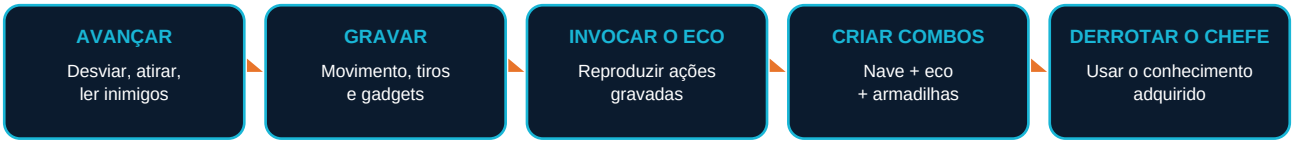
2. Formato e escopo

Item	Definição inicial
Gênero	Shooter 2D com padrões de bullet hell
Câmera	Lateral, com avanço horizontal
Plataforma-alvo	PC
Motor	Unity 2D
Modo	Single-player
Estrutura	Runs curtas com encontros de combate e chefe
Duração-alvo da run	5–10 minutos no protótipo completo
Duração de uma gravação	5 segundos
Ecos simultâneos	Até 3 no protótipo
Conteúdo inicial	Uma arena, um conjunto de armadilhas e um chefe
Controles	Teclado e mouse ou teclado; suporte a controle pode vir depois

Os números são parâmetros iniciais de design para testar, não valores definitivos.

3. Core gameplay loop

O jogo gira em torno de aprender os padrões, gravar ações úteis e usá-las para criar oportunidades de ataque.



4. Sistema principal — Temporal Echo

4.1. O que o jogador grava

O sistema registra uma sequência de ações da nave em uma janela temporal de cinco segundos.

Dado	Comportamento
Posição	Registra a posição da nave ao longo do tempo
Rotação	Registra a orientação da nave
Disparos	Registra instante, tipo e direção dos tiros
Gadgets	Registra o momento e a direção de uso
Duração	Cinco segundos por gravação
Reprodução	Uma cópia independente repete a sequência
Limite	Até três ecos ativos no protótipo

A gravação não precisa ser um replay de vídeo. Ela é uma lista de dados que permite reproduzir os eventos da nave.

4.2. Como criar o eco

O jogador aciona a habilidade temporal. A gravação selecionada é reproduzida por uma nave fantasma, que começa no ponto inicial registrado e executa a sequência na mesma ordem.

Durante a reprodução, o jogador mantém o controle da nave atual e pode criar uma nova sequência.

> **Regra fundamental:** o eco reproduz as ações, mas não restaura o estado anterior do mundo. Inimigos mortos não voltam à vida, projéteis já destruídos não reaparecem e armadilhas continuam seguindo suas regras atuais.

Isso evita a complexidade de rebobinar toda a simulação e torna o sistema mais previsível.

4.3. Ciclo de gravação

Para o protótipo, o sistema proposto é simples:

1. O jogador mantém pressionado o botão de gravação.
2. O jogo armazena até cinco segundos de movimento e eventos.
3. Ao soltar o botão, a sequência fica pronta.
4. O jogador aciona o eco para reproduzi-la.
5. Após a reprodução, o eco desaparece, a menos que uma habilidade permita mantê-lo ativo.

A gravação deve ter um indicador visual de tempo restante, para que o jogador consiga planejar suas ações sem olhar constantemente para a interface.

4.4. Limites e regras de consistência

- O eco não toma decisões e não corrige a trajetória automaticamente.
- A reprodução não muda de direção para perseguir inimigos.
- Os tiros seguem a direção registrada.
- O eco não causa colisão física com a nave atual.
- O eco não recebe dano de projéteis comuns no protótipo.
- Gadgets reproduzidos precisam obedecer às regras de cada gadget.
- O limite de ecos deve evitar que a tela fique ilegível.

Uma decisão importante para a versão final é como lidar com dispositivos que exigem presença contínua. Um eco pode manter um interruptor ativado enquanto sua trajetória permanece sobre ele, mas não deve ativar um mecanismo simplesmente por passar perto, a menos que essa seja a regra explícita do objeto.

5. Combate

5.1. Nave do jogador

A nave começa com um conjunto pequeno de ações:

- Movimento em oito direções dentro da área jogável.
- Disparo principal contínuo.
- Gadget equipado.
- Habilidade de gravação e reprodução temporal.
- Vida limitada e invulnerabilidade breve após receber dano.

O jogador deve conseguir atirar enquanto se movimenta. O foco é esquivar e posicionar-se, não alternar constantemente entre movimento e ataque.

5.2. Tipos de dano

Para simplificar o protótipo, existem apenas três categorias:

- **Dano direto:** tiros convencionais da nave.
- **Dano de gadget:** minas, pulsos ou ataques especiais.
- **Dano combinado:** efeitos especiais ativados quando as ações da nave atual e do eco coincidem.

5.3. Combos temporais

Os combos são eventos específicos reconhecidos pelo jogo, não apenas multiplicadores genéricos.

Combo	Condição	Resultado
Crossfire	Nave atual e eco acertam o mesmo alvo em uma janela curta	Dano adicional
Trap Bait	Eco atrai inimigos para uma armadilha	Inimigos atingidos pela armadilha
Double Pulse	Dois pulsos atingem mecanismos conectados	Sobrecarga ou desativação
Reflect Chain	Um projétil ricocheteia em escudos posicionados em sequência	Dano elevado ao alvo
Synchronized Strike	Dois pontos fracos são atingidos quase simultaneamente	Escudo do chefe desativado

Cada combo precisa de um feedback visual e sonoro próprio. O jogador deve entender por que a combinação funcionou.

5.4. Pontuação por habilidade

O jogo pode premiar:

- Combos temporais.
- Inimigos eliminados por armadilhas.
- Sequências sem sofrer dano.
- Sincronizações precisas.
- Destruição de projéteis ou mecanismos usando o ambiente.

A pontuação não deve ser necessária para terminar a fase, mas pode incentivar o domínio da mecânica e a rejogabilidade.

5.5. Inimigos e ondas

Os inimigos surgem durante o avanço da fase, entrando pela direita (e, em alguns casos, por cima ou por baixo). Em vez de spawn aleatório, cada encontro segue uma lista de ondas com tempo e posição definidos. Assim o jogador aprende o padrão e consegue planejar o que gravar no eco.

Tipos de inimigo

Para a jam, a recomendação é começar com um tipo e adicionar os outros dois somente se o núcleo já estiver divertido.

Inimigo	Comportamento	Papel com o eco
Drone (reto)	Entra pela direita e voa em linha reta; não atira	Alvo fácil para Crossfire; ensina a gravação
Ondulante	Move-se em zigue-zague senoidal, em grupos de 3 a 5	Pode ser conduzido para armadilhas (Trap Bait)
Torreta	Fica parada no cenário e atira em direção à nave	Exige que nave e eco ataquem de posições diferentes

Regras de surgimento

- As ondas são definidas por script: tempo de entrada, ponto de entrada, tipo e quantidade.
- Cada onda sinaliza a entrada com um aviso visual curto antes de aparecer (seta ou brilho na borda da tela).
- Inimigos que saem da tela pela esquerda são removidos sem penalidade.
- O número de inimigos simultâneos fica limitado, para a tela continuar legível com os ecos ativos.
- Matar inimigos nunca é obrigatório para avançar; o scroll continua independentemente.

Exemplo de linha do tempo — Encontro 1 (primeiro ciclo)

Tempo	Onda	Objetivo de design
0:00 – 0:10	Sem inimigos; apenas obstáculos básicos	Aprender movimento e tiro
0:10 – 0:25	3 drones em fila, vindo do centro	Primeiros alvos, sem pressão
0:25 – 0:40	Drones em dois níveis de altura	Incentivar gravar uma rota e deixar o eco cobrir um nível
0:40 – 1:00	Primeiro grupo de ondulantes	Ler o padrão e atacar com nave e eco ao mesmo tempo
1:00 – 1:20	Torreta + drones	Introduzir a ideia de dividir tarefas entre nave e eco

Os tempos acima são valores de partida para testar, não definitivos. As ondas dos encontros 2 e 3 reaproveitam os mesmos três inimigos, combinados com as armadilhas e o gadget.

6. Armadilhas e obstáculos

As armadilhas são parte central do jogo. Elas precisam desafiar a movimentação e também permitir que o jogador crie soluções com os ecos.

Laser gate

Barreiras que ligam e desligam em ciclos. O jogador precisa observar o padrão e atravessar no momento seguro.

Interação temporal: o eco ativa um interruptor que desliga uma barreira enquanto a nave atual atravessa outra.

Laser rotativo

Um feixe varre a área jogável e obriga o jogador a mudar de altitude.

Interação temporal: o eco atrai inimigos para a linha de fogo ou repete uma sequência de ativação de escudos.

Campo de minas

Minas explodem ao contato ou após um gatilho.

Interação temporal: a nave atual dispara um detonador quando os inimigos passam pela região registrada pelo eco.

Barreiras móveis

Obstáculos que abrem e fecham rotas, exigindo precisão de movimento.

Interação temporal: o eco mantém um mecanismo em determinado estado, abrindo uma janela de passagem.

> **Para a jam,** implemente apenas dois tipos de armadilha e uma variação de obstáculo. A variedade deve vir das combinações com o eco, e não da quantidade de objetos diferentes.

7. Gadgets

O jogador pode carregar um gadget por vez no protótipo. O objetivo é criar escolhas táticas sem sobrecarregar os controles.

Gadget	Função principal	Interação temporal
Mina temporal	Explode por proximidade ou detonação	O eco reproduz a colocação de minas
Escudo refletor	Bloqueia ou reflete determinados tiros	Permite construir uma sequência de ricochetes
Pulso de interferência	Desativa temporariamente lasers e sensores	Duas ativações podem sincronizar mecanismos
Arpão magnético	Puxa a nave ou inimigos leves	Pode conduzir inimigos para uma armadilha

Escopo inicial: mina temporal e pulso de interferência. Escudo refletor e arpão ficam para expansões, caso o núcleo já esteja divertido.

8. Puzzles integrados ao combate

Os puzzles devem surgir naturalmente durante o voo e durar poucos segundos. Eles funcionam como desafios de leitura, sincronização e execução.

Puzzle 1 — Dois interruptores

Objetivo: atravessar um corredor com dois lasers ligados a interruptores em pontos diferentes.

Solução esperada:

1. Gravar uma passagem pelo primeiro interruptor.
2. Reproduzir o eco para repetir essa ativação.
3. Pilotar a nave atual até o segundo interruptor.
4. Atravessar o corredor com as duas barreiras desativadas.

O jogador aprende a dividir uma tarefa entre duas versões da nave.

Puzzle 2 — Refletir o ataque do chefe

Objetivo: atingir um núcleo protegido, usando o ataque do próprio chefe.

1. O chefe dispara um projétil pesado.
2. O jogador grava uma sequência que posiciona um escudo refletor.
3. O eco reproduz a colocação da barreira.
4. O jogador se posiciona para induzir o disparo na direção correta.
5. O projétil ricocheteia e atinge o núcleo.

Essa versão depende da implementação do escudo refletor e pode ficar fora do protótipo inicial.

Puzzle 3 — Sincronização de energia

Objetivo: ativar dois receptores dentro de uma janela curta.

O eco ativa um receptor durante a reprodução, enquanto a nave atual ativa o outro. Se os dois estiverem ativos ao mesmo tempo, a porta ou a defesa do chefe é desativada.

O puzzle deve ter feedback visual claro: os receptores acendem, uma conexão de energia aparece e o mecanismo muda de estado.

9. Chefes

O protótipo deve ter apenas um chefe, projetado para demonstrar o potencial da mecânica temporal.

Chefe do protótipo — O Guardião do Ciclo (nome provisório)

Uma máquina de combate que protege uma passagem temporal. O núcleo permanece blindado durante a maior parte do encontro.

Fase 1 — Sobrevivência

O Guardião dispara projéteis em padrões simples e varre a tela com um laser. O jogador aprende a se movimentar, identificar janelas seguras e gravar uma rota.

Fase 2 — Preparação

Dois geradores laterais protegem o núcleo. O jogador pode gravar uma sequência de tiros contra um gerador e atacar o outro enquanto o eco repete a ação.

Fase 3 — Sincronização

Os dois geradores precisam ser destruídos dentro de uma janela curta. Quando isso acontece, o núcleo fica vulnerável.

O jogador deve coordenar os disparos da nave atual com os do eco, mantendo-se vivo durante o padrão de projéteis.

Fase 4 — Última chance

Com pouca vida, o Guardião aumenta a frequência dos ataques e reduz as janelas de vulnerabilidade. O jogador precisa aproveitar os ecos já criados para concluir o combate.

> **Critério de sucesso:** o chefe precisa ser vencível com uma boa sequência temporal, mas não deve exigir que o jogador execute uma combinação pixel-perfect. O desafio deve vir da leitura de padrões e do planejamento.

10. Estrutura das runs

O jogo usa runs curtas, com encontros que introduzem e combinam as mecânicas.

Encontro	Conteúdo	Objetivo de design
1. Primeiro ciclo	Inimigos simples e obstáculos básicos	Ensinar movimentação, tiro e gravação
2. Armadilha	Laser e interruptor	Ensinar interação entre eco e cenário
3. Combinação	Inimigos, armadilha e gadget	Ensinar combos temporais
4. Guardião	Chefe com duas fases de combate	Testar domínio da mecânica

No protótipo, os encontros podem ser segmentos de uma única fase, sem necessidade de um mapa de progressão completo.

Progressão entre tentativas

A estrutura de repetição deve reforçar o tema One More Time. Entretanto, é importante decidir qual modelo será usado:

- **Modelo de eco durante a vida:** o jogador grava e reproduz sequências enquanto está vivo. A morte reinicia o encontro.
- **Modelo de ciclos acumulados:** ao reiniciar, o jogador deixa um eco persistente de sua tentativa anterior.

Para o protótipo, a recomendação é o primeiro modelo como sistema principal, porque é mais previsível e fácil de balancear. O segundo pode ser uma evolução futura, caso a ideia de "tentar mais uma vez" também seja a estrutura narrativa do jogo.

11. Controles e interface

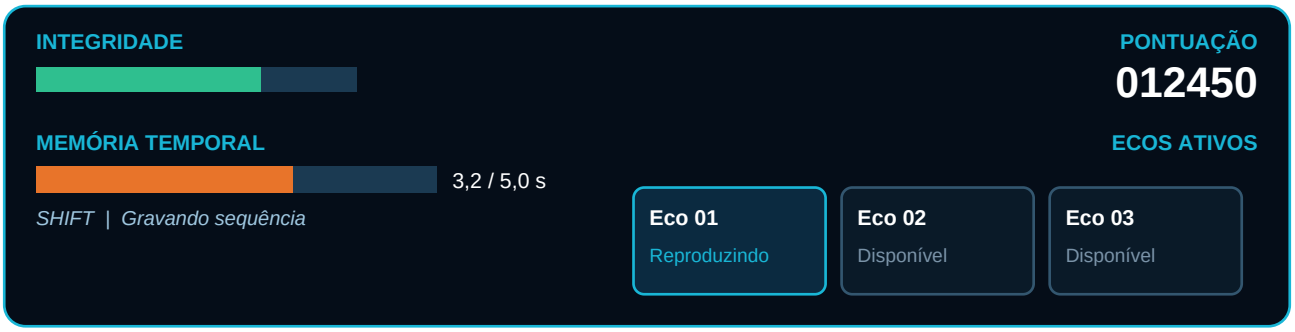
Os controles precisam permitir que o jogador continue desviando e atirando enquanto gerencia a habilidade temporal.

Controles propostos

Ação	Teclado
Movimentar nave	WASD ou setas
Disparo principal	Espaço ou botão esquerdo do mouse
Gravar sequência	Segurar Shift
Invocar eco	E
Usar gadget	Q
Pausar	Esc

Esses atalhos são provisórios. O importante é separar a gravação da reprodução para que o jogador não invoque o eco acidentalmente enquanto tenta desviar de uma armadilha.

HUD



Mockup conceitual de HUD — não representa uma interface implementada.

O HUD deve mostrar claramente:

- Vida restante.
- Estado e duração da gravação.
- Quantidade de ecos ativos.
- Gadget equipado e tempo de recarga.
- Vida do chefe e indicação de vulnerabilidade.
- Feedback quando um combo temporal é ativado.

12. Direção visual e sonora

Direção visual

Estética de arcade sci-fi com silhuetas legíveis, efeitos de energia e contrastes fortes.

- **Nave atual:** azul ou ciano intenso, com contorno nítido e disparos bem visíveis.
- **Eco temporal:** transparente, com contorno luminoso e um rastro que sugere repetição. Deve ser distinguível da nave atual mesmo durante muitos projéteis.
- **Inimigos e perigos:** laranja e vermelho, com sinais visuais claros antes de cada ataque.

Direção sonora

O áudio também pode comunicar a mecânica temporal:

- **Gravação:** pulso eletrônico que aumenta de frequência.
- **Invocação do eco:** som de reversão seguido de um impacto digital.
- **Reprodução:** versão filtrada dos sons da nave original.
- **Combo temporal:** efeito sonoro exclusivo e curto.
- **Chefe vulnerável:** mudança musical ou sinal sonoro marcante.

Um detalhe interessante seria fazer os disparos do eco soarem levemente abafados, como se viessem de outra camada temporal.

13. Arquitetura técnica sugerida para Unity

A implementação deve ser modular para que o sistema temporal não dependa diretamente da lógica de cada inimigo ou armadilha.

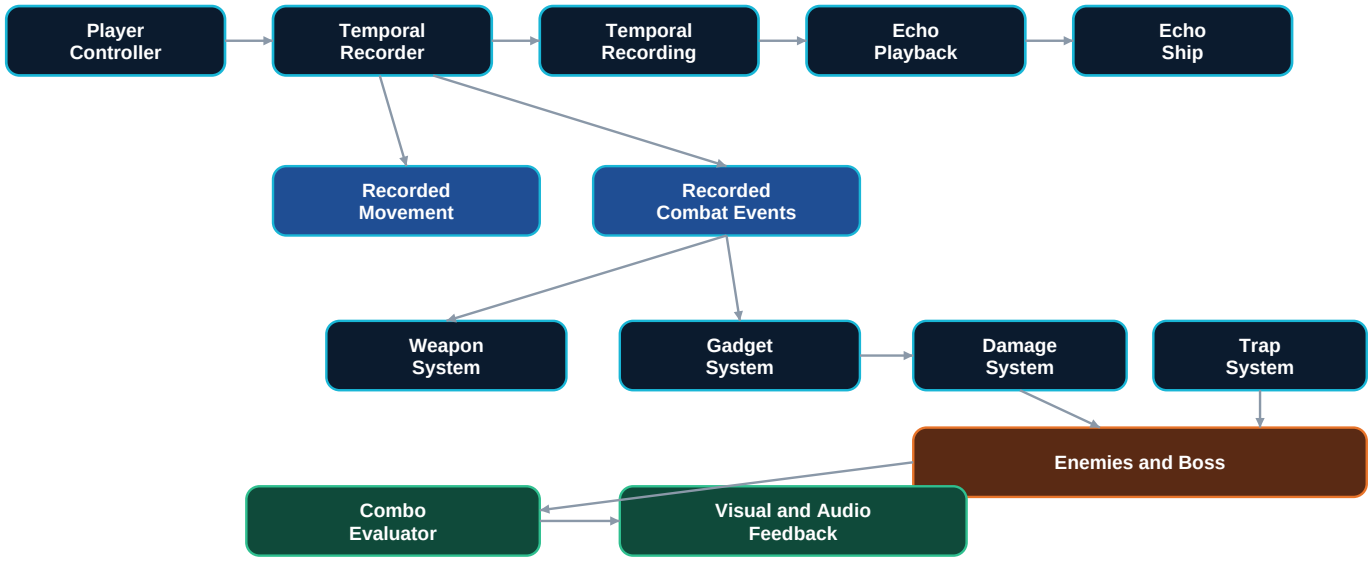


Diagrama conceitual redesenhado para este PDF.

Componentes principais

Componente	Responsabilidade
`PlayerController`	Movimento e entrada do jogador
`TemporalRecorder`	Captura das amostras e dos eventos
`TemporalRecording`	Armazena a sequência gravada
`EchoPlayback`	Reproduz a sequência em ordem temporal
`WeaponSystem`	Gerencia disparos e projéteis
`GadgetSystem`	Ativa e registra gadgets
`TrapController`	Controla lasers, interruptores e armadilhas
`EnemyController`	Movimento e comportamento dos inimigos
`BossController`	Fases, ataques e pontos fracos
`ComboEvaluator`	Detecta condições de combos temporais

Como armazenar a gravação

Uma amostra de movimento pode conter:

```
public struct EchoSample
{
    public float time;
    public Vector2 position;
    public float rotation;
}
```

Os disparos e gadgets ficam em uma lista separada de eventos, cada um com seu instante relativo à gravação.

```
public struct EchoEvent
{
    public float time;
    public EchoEventType type;
    public Vector2 position;
    public Vector2 direction;
}
```

`EchoEventType` seria um enum com os tipos de evento que o protótipo suporta, como disparo e uso de gadget.

> **Importante:** não é necessário armazenar uma amostra a cada frame. Uma taxa fixa, por exemplo 20 amostras por segundo, costuma ser suficiente para começar. A reprodução pode interpolar as posições entre amostras. Os eventos de disparo e ativação precisam respeitar seus instantes originais.

Para evitar erros, a reprodução deve usar o tempo decorrido do próprio eco e disparar cada evento uma única vez. Os projéteis devem nascer no ponto e na direção registrados, mas suas colisões serão avaliadas no estado atual do mundo.

14. Escopo da game jam

O objetivo é validar se a mecânica central é divertida, não construir todo o conteúdo previsto para o jogo final.

Obrigatório — MVP

- Nave com movimentação e disparo.
- Uma arena side-scrolling.
- Gravação de cinco segundos.
- Reprodução de movimento e disparos.
- Pelo menos um tipo de inimigo (drone), com ondas por script.
- Duas armadilhas.
- Um gadget funcional.
- Um chefe com uma janela de vulnerabilidade temporal.
- Interface mínima e condição de vitória e derrota.

Opcional — somente após validar o MVP

- Até três ecos simultâneos.
- Um combo temporal com feedback próprio.
- Um puzzle de sincronização.

- Um segundo gadget.
- Pontuação por combos e esquivas.

Fora do escopo inicial

- Rebobinar o mundo inteiro.
- Campanha extensa.
- Vários chefes.
- Progressão roguelite complexa.
- Árvore de habilidades.
- Muitos gadgets com sistemas de física independentes.

15. Critérios de aceitação do protótipo

O protótipo estará pronto para teste quando:

1. O jogador conseguir gravar e reproduzir uma sequência de cinco segundos.
2. O eco repetir movimentos e disparos sem depender do controle atual.
3. A reprodução continuar funcionando quando os inimigos se moverem ou morrerem.
4. Uma armadilha puder ser manipulada por uma ação registrada.
5. O jogador conseguir combinar o eco com a nave atual para criar pelo menos um ataque especial.
6. O chefe puder ser derrotado usando a mecânica temporal.
7. O jogador entender visualmente qual nave é a atual e qual é o eco.
8. A mecânica continuar legível durante momentos de maior quantidade de projéteis.

> O teste mais importante é observar alguém jogar sem explicar a solução: a pessoa deve perceber que pode gravar uma ação e aproveitar sua repetição para realizar outra.

16. Riscos de design e soluções

Risco	Mitigação
O jogador não entende para que serve o eco	Criar um encontro inicial em que a solução exige claramente duas ações simultâneas
Os ecos deixam a tela confusa	Limitar a quantidade, reduzir opacidade e distinguir os efeitos visuais
A gravação exige precisão excessiva	Usar janelas de sincronização generosas no começo
Os combos viram apenas bônus de dano	Criar interações com armadilhas, escudos e pontos fracos
O sistema fica complexo de implementar	Reproduzir somente dados e eventos do jogador, sem rebobinar a cena inteira
O chefe parece injusto	Sinalizar os ataques, as janelas de vulnerabilidade e o resultado de cada combo

17. Resumo executivo

ONE MORE TIME é um shooter 2D de nave em que o jogador grava e reproduz suas ações passadas para criar ataques coordenados, superar armadilhas e derrotar chefes.

O diferencial está em transformar o próprio histórico do jogador em uma ferramenta de gameplay. Uma esquiva pode se tornar uma distração, um disparo pode preparar um ataque posterior e um gadget pode abrir uma rota que seria impossível atravessar sozinho.

O protótipo deve concentrar-se em três experiências:

- **Repetir:** observar o eco executando exatamente o que foi gravado.
- **Combinar:** usar a nave atual e o eco para atacar, desviar ou manipular armadilhas.
- **Descobrir:** perceber uma solução temporal para um desafio que parecia impossível.

Recomendação de produção: começar com uma única arena e um chefe que exija dois ataques sincronizados. Se essa experiência funcionar e o jogador sentir que realmente planejou uma solução inteligente, o coração do jogo está validado.

O restante — mais gadgets, chefes, upgrades e estrutura roguelite — poderá crescer a partir desse núcleo, sem colocar em risco a entrega da game jam.